Company Brain — Architecture

Oct 14, 2026 · @Atharva Kulkarni

At a glance

Company Brain is a permission-aware knowledge graph on FalkorDB, fed by Slack, Google Drive, GitHub and Notion, that agents query through MCP to answer questions, draft work and raise alerts, with every claim traced to its source.

The demo company has two departments, Sales and Engineering, so the graph has to show how one affects the other. Three flows carry the demo: Cursor recommends the API the team agreed on, a Sales email draft reflects the real engineering status, and a reversed engineering decision alerts the Sales owners it hurts.

Design principles

1. Raw first, graph second. Every fetched item is stored untouched; graph nodes point to it, so we can re-extract, cite and retract.
2. Fixed core ontology with free-form properties. Predictable edge types make multi-hop queries and permission checks reliable.
3. No evidence, no fact. A Decision, Commitment or Figure without an `EVIDENCED_BY` edge is never shown.
4. Sources write the graph, agents do not. Only the ingestion pipeline writes the company graph, and only from items that carry ACLs, so a conversation can never turn a private fact into shared knowledge.
5. Supersede, never overwrite. History stays queryable; "current" is a filter on `status = active`.
6. Permissions are enforced in code at query time. The LLM never decides who may see what.
7. Agent memory points at the graph and never copies company facts, so it cannot go stale.
8. Agents call fixed, vetted tools, not free-form Cypher. Each tool has the permission filter built in.

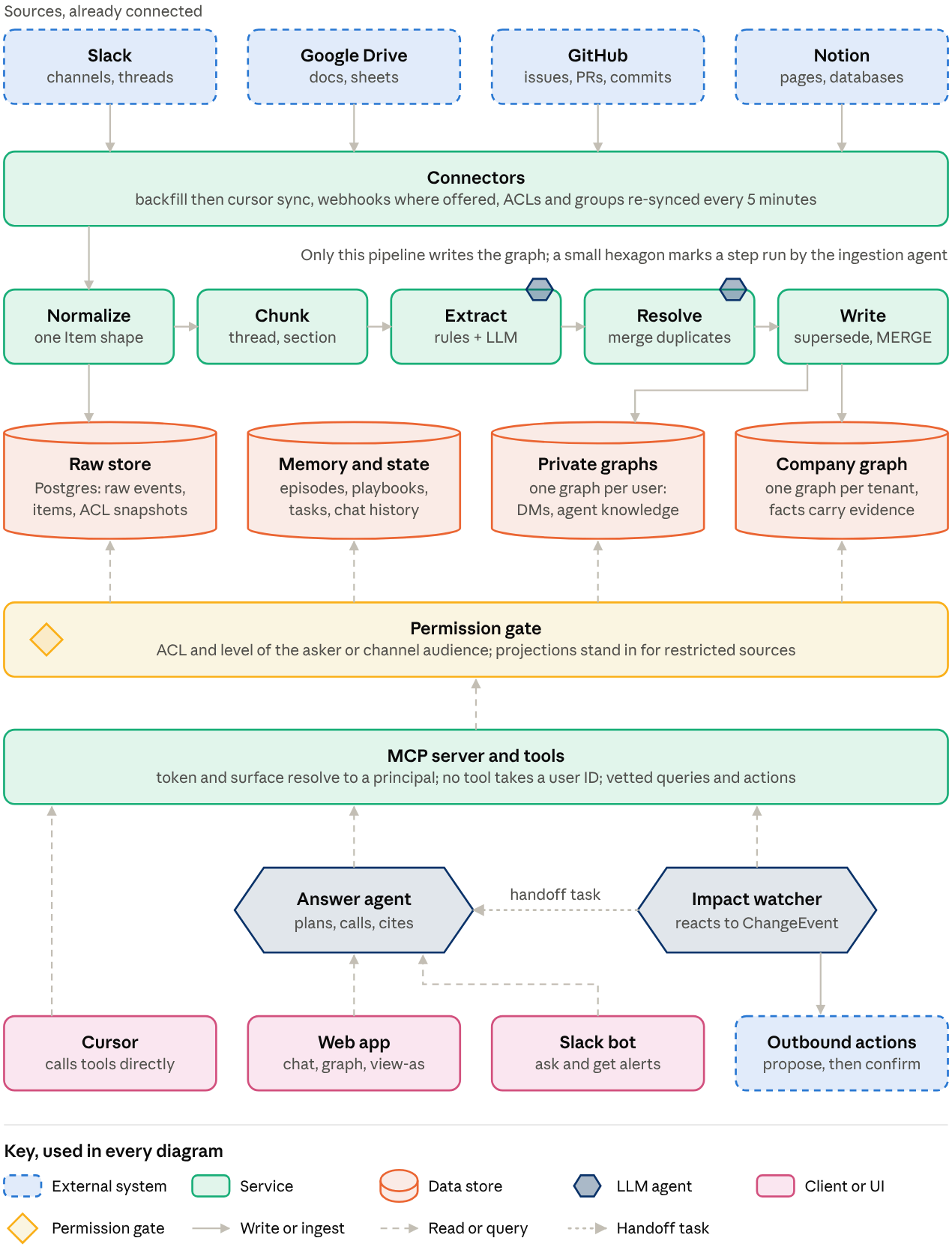
Assumptions to confirm

- Hierarchy: an intern's agent sees management-level *context*, meaning summaries of decisions that affect them. The raw management sources stay restricted. If you want interns to see raw sources, lower `min_level` on those sources.
- Stack (you have not told me yours): Python with FastAPI workers, FalkorDB, Postgres with pgvector for the raw store, history and state, an MCP server over HTTP, and any LLM that returns structured JSON.
- Email and Jira are out of v1. The finance-email example becomes pricing and delivery figures taken from Notion and Drive pages.
- Org chart (reporting line, level, department) comes from a seeded file, because none of the four tools holds it reliably.
- Outbound actions are propose-then-confirm, except internal Slack notices.

System overview

A single permission gate stands between every agent and every store, so no answer can use a source its caller may not see.

One permission gate sits between every agent and every store



system layers · 4 sources, 4 stores, 1 gate, 3 agents, 4 client surfaces

Read it top to bottom for ingestion (solid arrows) and bottom to top for questions (dashed arrows). The key at the foot applies to every diagram that follows.

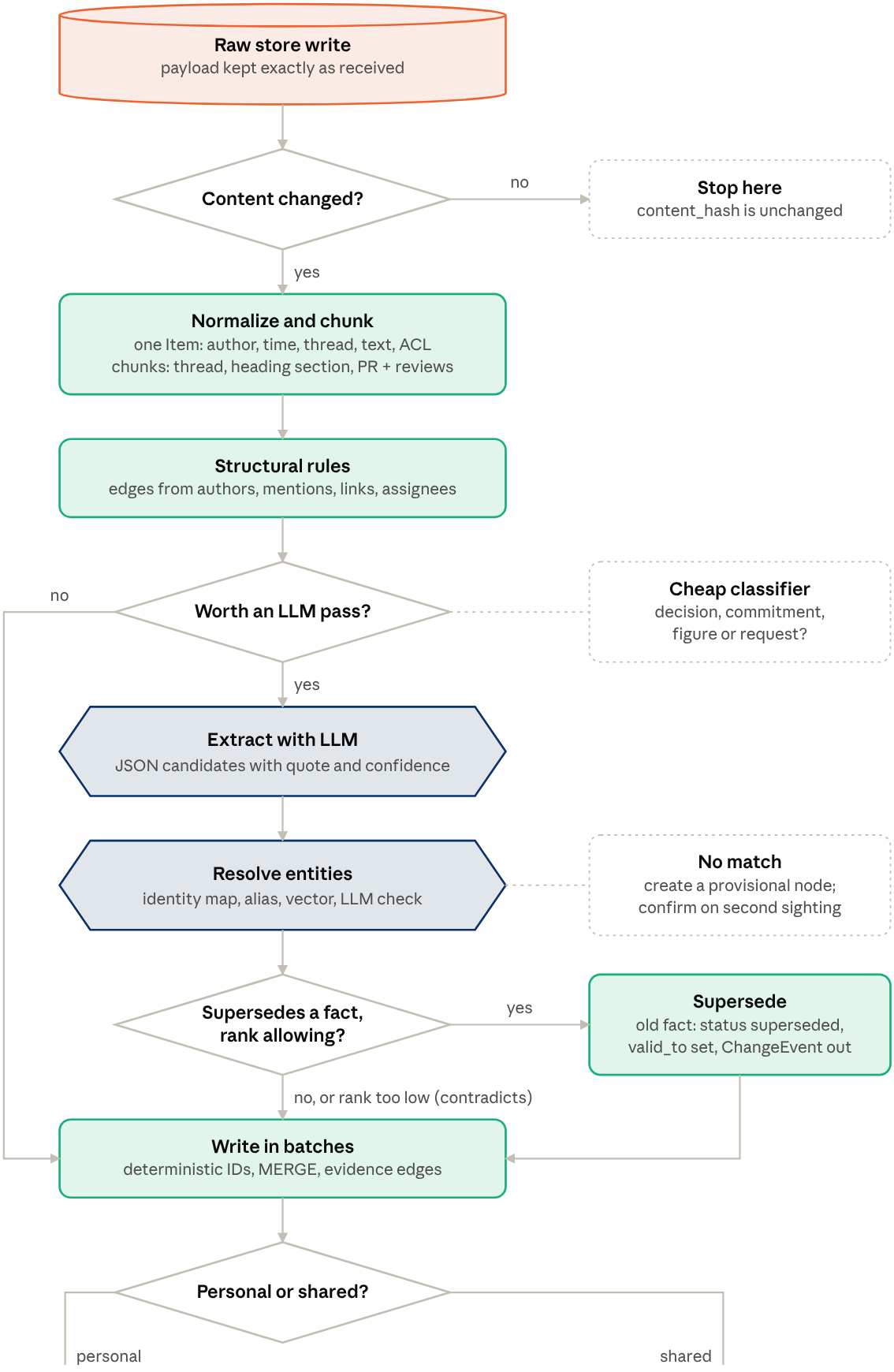
Component	Responsibility	Built with (assumed)
Connectors	Pull content, group membership and permissions from Slack, Drive, GitHub and Notion; backfill first, then incremental	Your existing connector code, with a cursor per source stored in Postgres
Pipeline workers	Normalize, chunk, extract, resolve, write; every step idempotent and retryable	Python workers on a Postgres job queue
Raw store	Untouched payloads, normalized items and ACL snapshots; the target of every citation	Postgres
Company graph	Entities, decisions, commitments, figures, evidence and projections; one graph per tenant	FalkorDB
Private graphs	Personal items and each user's agent knowledge; one graph per user	FalkorDB
Memory and state	Episodes, playbooks, conversations, tasks, audit log	Postgres with pgvector
Permission gate	Computes which items a caller may see from source ACL, level, groups and projections; applied inside every tool	One library shared by all tools
MCP server	Exposes the vetted tools and turns a token and surface into a principal	Python MCP SDK over HTTP
Answer agent	Intent routing, bounded tool loop, citation validation, drafting	LLM with structured output
Impact watcher	Consumes change events, walks dependencies, notifies owners	Worker plus LLM for drafting

Component	Responsibility	Built with (assumed)
Web app	Chat with the answer card, graph explorer, department roll-up, view-as switcher	Your choice of frontend

Ingestion pipeline

Nothing reaches the graph directly: every item is stored raw, normalized, extracted, resolved and checked against what is already true before it is written.

Nothing enters the graph until it is checked against what is already true





Unfilled diamond: a yes or no test. Other shapes follow the key in the system overview.

ingestion flow · 4 tests, 2 LLM steps, 2 target graphs

Solid arrows are the main path. The left-hand bypass skips the LLM for chunks the classifier rejects, and the dotted ties explain what each test decides.

#	Stage	What it does	How we build it	Emits
1	Fetch	Full backfill, then incremental changes, with ACL and group membership for each item	Cursor per source in `sync_state`; a webhook handler where the tool offers one (for Slack it verifies the signature, records the event ID, enqueues a job and answers inside Slack's 3-second limit), otherwise a polling job; ACL and group membership re-synced every 5 minutes	`raw_event`, `source_container`, ACL snapshot
2	Store raw	Keep the payload untouched	Insert keyed by system, external ID and `content_hash`; an unchanged hash ends the run	`raw_event` row
3	Normalize	One shape for every source	A mapper per source into `Item` with source, external ID, URL, author, time, thread, text and ACL	`item` row

#	Stage	What it does	How we build it	Emits
4	Chunk	Units that carry meaning	Slack: whole thread. Notion and Drive: split at headings, roughly 500 to 800 tokens. GitHub: description, review comments and linked issue together	`chunk` rows with embeddings
5	Structural rules	Free, precise edges	Author, @mentions, links between items, PR closes issue, assignee, channel to team	Edges with `confidence = 1`
6	Classifier gate	Skip chunks with nothing to extract	Small model, yes or no on decision, commitment, figure or request	Pass or skip flag
7	LLM extraction	Candidates with evidence	Structured JSON against the ontology; each candidate carries `quote`, `span`, `confidence` and subject strings; discussion is not a decision until someone commits	Candidate list

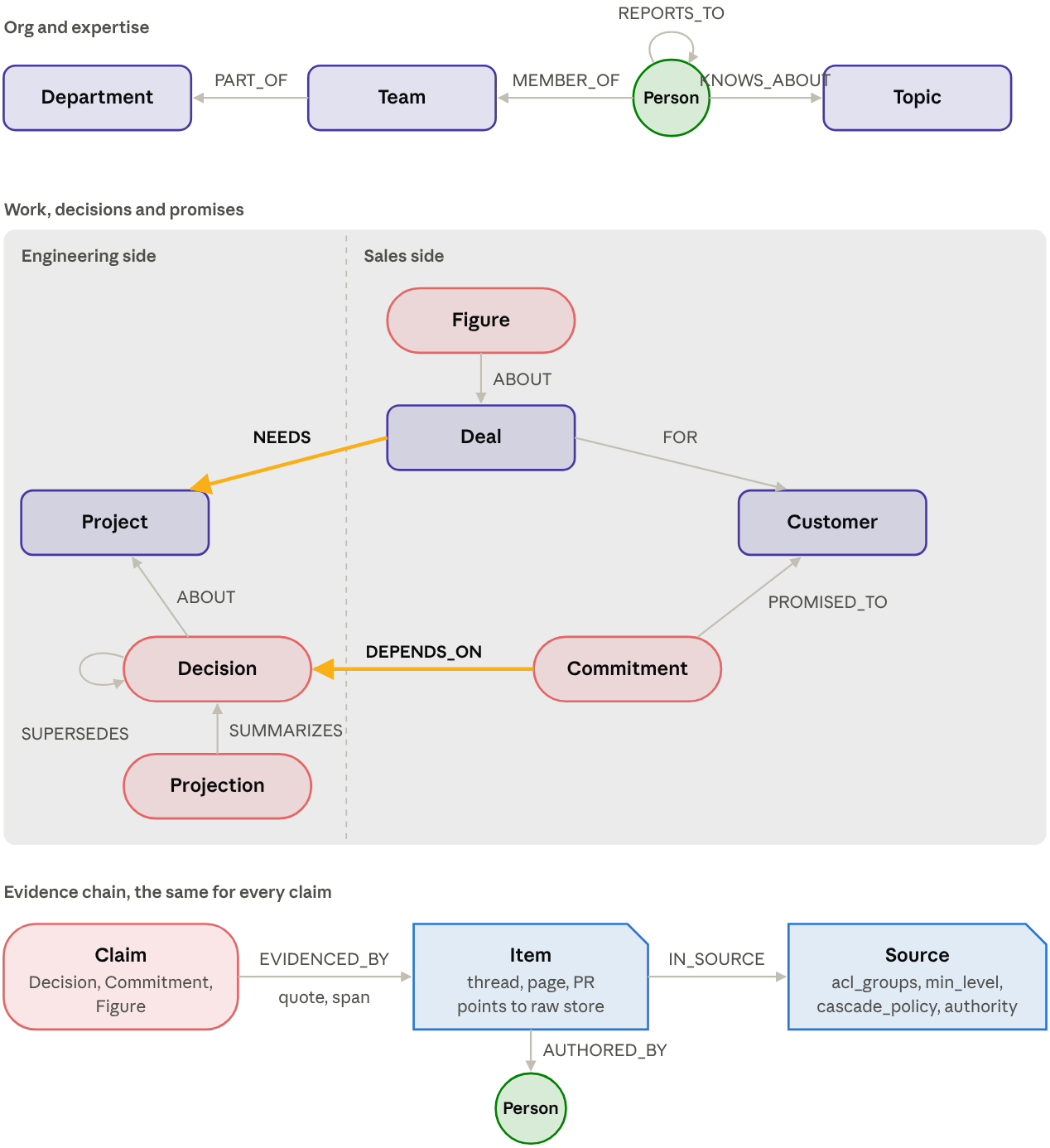
#	Stage	What it does	How we build it	Emits
8	Resolve	Strings become nodes	People: identity map keyed by email, covering Slack ID, GitHub login and Notion user. Topics, projects, customers: alias table, then vector match, then an LLM check in the gray zone. No match creates a provisional node, confirmed on a second sighting	Node IDs, `resolution_log`
9	Supersede	Keep "current" correct	Compare with active facts on the same subject, rules first and LLM adjudication for the rest; outcomes are duplicate, supersedes, contradicts or coexists; a newer claim from a source that ranks lower in authority than the active claim's cannot supersede it and is stored as contradicts, unless its author made the old claim	`SUPERSEDES` edge, `ChangeEvent`

#	Stage	What it does	How we build it	Emits
10	Write	Persist	Deterministic IDs, `MERGE` inside `UNWIND` batches, target graph chosen from the item's tier. The `ChangeEvent` goes to a Postgres outbox right after the graph write succeeds; retries are safe because writes are idempotent and events are keyed by fact ID	Graph writes, outbox row
11	After write	Derived data	Nightly `KNOWS_ABOUT` recompute; ACL re-sync every 5 minutes; a reconcile pass that finds items deleted at the source	Scores, tombstones

Graph data model

A fixed core of thirteen node types and a small set of typed edges keeps multi-hop queries predictable, and the Sales-to-Engineering dependency lives in two edge types, `NEEDS` and `DEPENDS_ON`.

Two edge types carry the dependency between Sales and Engineering



graph schema · 13 node types in 4 families, 2 bridge edges between departments

The highlighted edges cross the department line. Every claim at the bottom carries `EVIDENCED_BY`, and the evidence chain runs from the claim to the item to the source, which holds the permissions. Edge types not drawn, such as `OWNS` and `MADE_BY`, are in the tables below.

Node types

Node	Meaning	Key properties
Person	Employee or contact	`email`, `level` (0 to 3), `dept`, identities (Slack ID, GitHub login, Notion user)
Team, Department	Org units	`name`, `kind`
Project	Feature, epic, repo or initiative	`name`, `status`, aliases
Topic	Technology, API, library or concept	`name`, aliases, embedding
Customer, Deal	Sales entities	`name`, `stage`, `renewal_date`
Decision	A choice that was made	`statement`, `status`, `valid_from`, `valid_to`, `confidence`
Commitment	A promise with a date	`statement`, `due`, `status`
Figure	A number about something	`name`, `value`, `unit`, `as_of`, `status`
Projection	Sanitized summary cascaded to a wider audience	`summary`, `audience`, `level_floor`, `origin_id`
Item	Normalized source object	`raw_id`, `url`, `time`, `acl_groups`, `min_level`
Source	Container that holds permissions	`system`, `acl_groups`, `min_level`, `cascade_policy`, `authority`

Edge types

Edge	From to	Notes
`MEMBER_OF`, `PART_OF`, `REPORTS_TO`	Person to Team, Team to Department, Person to Person	From the seeded org chart
`OWNS`	Person to Project, Commitment, Topic	Used to find who to notify
`KNOWS_ABOUT`	Person to Topic, Project	`score` from authorship and replies with recency decay; answers "who knows what"
`ABOUT`	Decision, Figure to Topic, Project, Deal	Subject of the claim
`MADE_BY`	Decision to Person	Decider
`SUPERSEDES`	Decision to Decision, Figure to Figure	Builds the history chain
`NEEDS`	Deal to Project	Bridge: Sales needs something Engineering builds
`DEPENDS_ON`	Commitment to Decision, Project, Commitment	Bridge when the two owners sit in different departments
`FOR`, `PROMISED_TO`	Deal to Customer, Commitment to Customer	Who the work is for
`BLOCKED_BY`	Project to Project, Decision	Delivery risk
`EVIDENCED_BY`	Decision, Commitment, Figure to Item	`quote`, `span`, `confidence`
`IN_SOURCE`, `AUTHORED_BY`, `MENTIONS`, `LINKS_TO`	Item to Source, Person, Topic or Project or Customer, Item	Structural edges from the rules step
`SUMMARIZES`	Projection to Decision, Commitment, Figure	Cascade to a wider audience

Time and supersession. Every claim has `status` (active, superseded, retracted), `valid_from`, `valid_to`, `last_verified_at` and `extractor_version`. "Current" means `status = 'active'`. A reversal adds the new claim, links it with `SUPERSEDES`, closes the

old one and emits a `ChangeEvent`. Old claims stay queryable, which is how the answer card can say what a decision replaced. A claim from a lower-authority source cannot supersede a higher-authority one unless its author made the old claim; it is stored as a contradiction and shown side by side.

Query sketches (openCypher style; check syntax against the FalkorDB docs). The tool adds the visibility filter, so the agent never writes it.

Current decision on a topic, with only the evidence the caller may see:

```
MATCH (d:Decision {status: 'active'})-[:ABOUT]->(t:Topic {id: $topic})
MATCH (d)-[e:EVIDENCED_BY]->(i:Item)
WHERE i.min_level <= $level AND ANY(g IN i.acl_groups WHERE g IN $groups)
RETURN d, collect({quote: e.quote, url: i.url, at: i.time}) AS evidence
ORDER BY d.valid_from DESC LIMIT 1
```

What a reversed decision touches, up to three hops away, with owners:

```
MATCH (old:Decision {id: $old_id})<-[:DEPENDS_ON|NEEDS*1..3]-(x)
OPTIONAL MATCH (p:Person)-[:OWNS]->(x)
RETURN x, labels(x) AS kind, p
```

Department roll-up of open cross-team dependencies:

```
MATCH (p1:Person)-[:OWNS]->(a:Commitment)-[:DEPENDS_ON]->(b)<-[:OWNS]-
(p2:Person)
WHERE a.status = 'open' AND p1.dept <> p2.dept
RETURN p1.dept AS from_dept, p2.dept AS to_dept, count(*) AS
open_dependencies
```

Visibility and hierarchy

A caller sees a fact only if at least one of its evidence items passes the source rule; management-level decisions reach lower levels as summaries, never as raw sources; and in a shared Slack channel the caller is the channel audience, not the asker.

Raw access needs both a group and a level; lower levels get summaries

	Sam L0 Eng intern	Dev L1 Eng	Priya L1 Sales	Maya L2 Eng lead	Rohan L3 Sales head
Public eng channel org-wide, any level	●	●	●	●	●
Eng team repo group eng, any level	●	●	○	●	○
#eng-leads group eng-leads, level 2+	○	○	○	●	○
#leadership group leadership, L2+, broadcast	◐	◐	◐	●	●
Leadership budget sheet group leadership, L3+, manual	○	○	○	○	●

● Raw access with quotes and links ◐ Summary only, through a projection ○ Nothing, as if absent

Permission gate: four checks run in order

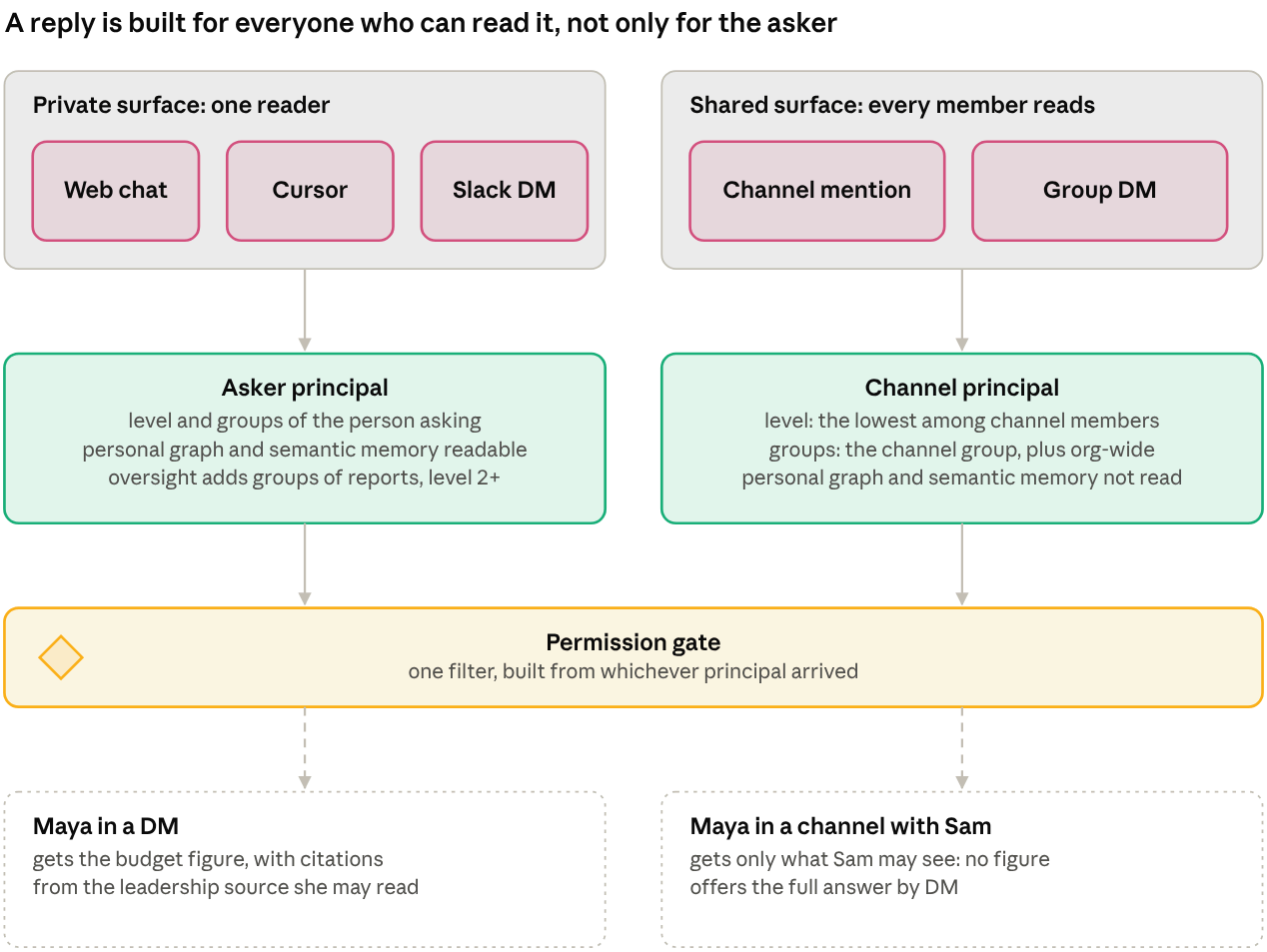
Source rule group match and level floor → **Oversight** managers add the groups of reports → **Claim rule** visible evidence item is enough → **Projection** audience match, summary only

Maya reaches the Eng team repo through her own membership and through oversight of her reports.

visibility matrix · 5 people, 5 sources, 4 gate checks

Read across a row to see who gets raw access, a summary or nothing. Level alone opens nothing: Rohan is level 3 but is not in the `eng-leads` group, so that channel stays closed to him. This is how an intern's agent sees what is happening at management level (the dotted half circles in the `#leadership` row) without seeing the leadership channel itself.

Who is the audience? A reply is built for everyone who can read it: the asker on a private surface, the channel on a shared one.



Solid: passes on. Dashed: result. Shapes follow the key in the system overview.

audience rule · 5 surfaces, 2 principals, 1 gate

Without this rule a manager asking in a busy channel would put management-level facts in front of interns. The bot never needs to know who is reading; it only needs the channel's member list, which the Slack sync already keeps. When the channel principal cannot see enough to answer, the bot says so and offers the full answer by DM.

The rules

1. **Principal.** A token resolves to `tenant`, `user`, `level` (0 intern, 1 individual contributor, 2 manager, 3 director), `dept` and `groups`. Groups come from source memberships (Slack channels, GitHub repo access, Drive sharing, Notion membership), mapped through the identity map.
2. **Audience rule.** Every request also carries a surface. On a private surface (web chat, Cursor over MCP, a Slack DM) the caller is the asker's principal. On a shared surface (a Slack channel or group DM) the caller is a channel principal: level is the lowest among

the channel members, groups are the channel's own group plus org-wide, and the personal graph and semantic memory are not read.

3. **Source rule.** An item is visible when its `acl_groups` overlap the caller's groups or it is org-wide, and the caller's level is at least the item's `min_level`.
4. **Oversight.** For level 2 and above, the caller's groups also include those of everyone below them in the `REPORTS_TO` tree.
5. **Claim rule.** A claim is visible when at least one of its evidence items is. Only visible evidence is returned, counted or ranked, so aggregates cannot leak.
6. **Projection rule.** A `Projection` is visible when the caller matches its audience. It shows the summary and names the source as restricted, with no quote and no link.

How we build it

- **Levels and policies.** None of the four tools knows our levels, so a small config table maps sources to `min_level` and `cascade_policy` (for example `#leadership` is level 2 and `broadcast`).
- **Source authority.** The same config table gives each source an `authority` rank: formal documents (Notion pages, Drive files) 3, trackers (GitHub issues and PRs) 2, chat (Slack) 1. Supersession reads it, as the ingestion pipeline explains.
- **One filter for all tools.** A library function turns a principal into a Cypher predicate plus parameters, and every tool uses it. The visibility matrix above becomes the test table: one test per person and source.
- **Projections.** When a `broadcast` source yields a decision, commitment or figure, the ingestion agent writes a two-sentence summary, a redaction pass strips figures and names flagged confidential, and the audience comes from the source's config. A `manual` source creates a draft that a manager approves in the web app before anyone sees it.
- **Search.** Vector and keyword search run only over chunks the caller can see; the filter is applied before ranking, and caches are keyed by the caller's visibility hash.
- **No hints.** Errors and empty results never mention restricted sources. The personal tier is read only for its owner and never merged into anyone else's results.
- **Shared surfaces.** The Slack connector already syncs channel membership. A helper turns a member list into a channel principal and caches it with the same `acl_epoch` as every other ACL snapshot. Answers in channels are built only with that principal, and the episode is stored with the channel as its audience.

- **Revocation.** ACL snapshots and group memberships re-sync every 5 minutes for all four sources, and Slack membership and privacy-change events refresh that channel at once. A revoked permission therefore takes effect within 5 minutes at worst, and usually immediately for Slack. If a source's last good sync is older than 15 minutes, the gate excludes that source, so a stalled sync hides data instead of leaking it. Visibility is checked at read time against each item's ACL, so a channel that flips between public and private needs no data moved. Cache keys include the tenant's `acl_epoch`, which rises whenever any snapshot changes, so no cached answer outlives a revocation.

To give an intern raw access to a management source instead, add the intern's group and lower its `min_level`; the gate needs no code change.

What we test. The visibility matrix is the first test table, and the new rules add the cases below. Each runs in CI against a small seeded graph and has to pass before the feature freeze.

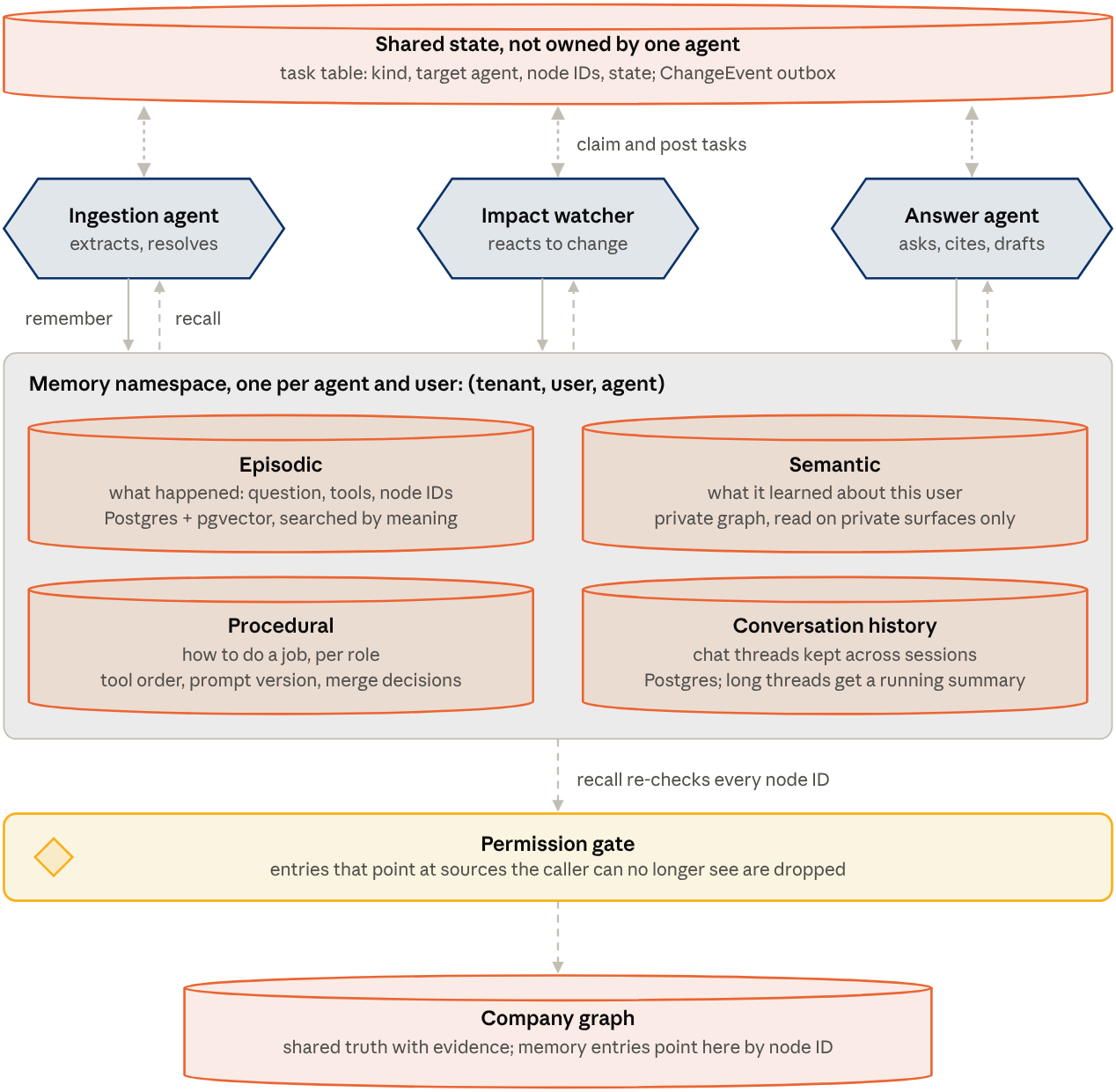
Test	Setup	Expected
Matrix	One test per person and source in the visibility matrix	Raw access, summary or nothing, exactly as drawn
Channel audience	Maya asks for the leadership budget figure in a channel Sam belongs to	No figure; only what Sam may see, plus an offer to answer by DM
Personal data in channels	A fact that exists only in the asker's private graph is asked about in a channel	Never returned, and semantic memory is not recalled
Privacy flip	A channel flips from public to private and the next sync runs	Non-members get no evidence from it
Revocation	A person leaves a group and the next sync runs	Their next query no longer returns that group's items, and no cached answer survives
Stale snapshot	A source's last good sync is older than 15 minutes	That source's items are excluded for everyone
Agents cannot write	Enumerate every registered tool and its query mode	No agent tool can issue a graph write; only the ingestion pipeline does
Slack retry	The same event is delivered twice	One raw event and one reply

Test	Setup	Expected
Authority	A Slack message contradicts an active decision recorded in a Notion page	Both shown side by side, and the old decision stays active

Agent memory

Each agent keeps its own memory in a namespace keyed by tenant, user and agent, and that memory holds pointers to the company graph, never copies of its facts.

Agents remember in their own namespace and point at the graph, never copy it



Tinted band: the memory namespace of one agent. Other shapes follow the key in the system overview.

agent memory · 4 memory kinds, 3 agents, 1 shared state, 1 gate

Solid arrows remember, dashed arrows recall, and dotted arrows hand tasks through shared state. Every recalled entry passes the permission gate again, so a memory cannot outlive the permission behind it.

Memory	What it holds	Lives in	Written	Example (answer agent)
Episodic	One row per answer or alert: question, tools called, node IDs, outcome, user feedback	Postgres with pgvector	After every answer and every alert	Recommended Provider A for SSO, citing the Slack thread; Sam accepted
Semantic	Stable facts about the user, each a link to graph IDs: repos, topics, who they usually ask	The user's private graph	Nightly consolidation, once a pattern repeats	Sam works on the SSO endpoint and asks Dev about it
Procedural	Playbooks per role: tool order, prompt and extractor versions, merge decisions the ingestion agent learned	Postgres, versioned	On deploy, or when a person approves a suggested change	For an API question: `get_decision`, check for a newer decision, then cite
Conversation history	Every thread, turn by turn, with a running summary for long threads	Postgres	Every turn	The thread where Priya asked for the Acme status email

How memory works

1. Remember. After each answer, the answer agent calls `remember` with an episode. When the user accepts, edits or rejects a draft, that feedback is added to the same episode.

2. Consolidate. A nightly job reads new episodes, merges repeated patterns into semantic entries in the user's private graph, and proposes playbook changes. A person approves each playbook change; nothing rewrites a playbook on its own.
3. Recall. Before planning, the agent calls `recall_memory` with the question. The best few entries come back with their node IDs, and the gate re-checks each ID. An entry whose source the caller can no longer see is dropped without a trace. On a shared surface semantic entries are not recalled at all, because they live in the personal graph.
4. Forget. A user can delete any of their entries in the web app. Deleting a user removes their private graph and their namespace rows.

Two rules keep memory honest. An entry stores a node ID and a short note, never a company fact, so a reversed decision shows up at the next recall. Namespaces never mix: Sam's answer agent cannot read Priya's, and the ingestion agent, which works for the whole tenant, has a namespace with no user.

Agents and MCP tools

Three agents share one set of vetted tools: an ingestion agent that writes the graph, an answer agent that reads it and drafts, and an impact watcher that reacts to change.

Agent	Triggered by	Reads	Writes	Own memory
Ingestion (curator)	Connector events	Raw store, graph (for resolution)	Graph, resolution log	Procedural: extraction prompt version, learned merge decisions
Answer ("Brain")	Web chat, Slack mention, MCP client such as Cursor	Graph through tools, its own memory	Conversation, episodes, drafts	Episodic and semantic per user, procedural playbooks per role
Impact watcher	`ChangeEvent` queue	Graph (impact traversal)	Notifications, tasks for the answer agent	Episodic log of alerts and how people responded

Handoffs go through a shared `task` table. A row holds the kind, the target agent, graph node IDs and a state. Agents claim rows with `FOR UPDATE SKIP LOCKED`, so two workers never take the same task, and the payload carries IDs, not copied text.

Write rule. Only the ingestion pipeline writes the company graph, and only from items that carry ACLs. The answer agent and the impact watcher get a graph client opened in read-only query mode, and their own writes go to their memory namespace, the task table and the audit log. A conversation can therefore never turn a private fact into shared knowledge. A test lists every registered tool and fails if any of them can issue a graph write.

Tool catalogue. Every tool resolves the caller from the token and the surface on the request. No tool accepts a user ID or a surface as a parameter; the surface comes from the signed Slack event or the MCP session.

Tool	Input	Returns	Permission handling
`search_context`	Query, optional filters	Hybrid vector and keyword hits expanded one hop, with citations	Items filtered by `visible(item)`
`get_decision`	Topic or ID, optional repo, optional as-of date	Active decision, its supersession chain, evidence	Evidence filtered; decision hidden if no visible evidence and no projection
`trace_decision`	Decision ID	Full path: decision, evidence items, authors, threads	Same filter; hidden hops are counted, not named
`find_owner`	Project, topic or customer	Owners, plus top people by `KNOWS_ABOUT` score with evidence	Scores computed over visible items only
`get_figures`	Subject, optional figure name	Latest figures with as-of date, source, history	Evidence filter

Tool	Input	Returns	Permission handling
`get_dependencies`	Entity, depth, direction	Neighbours over `DEPENDS_ON`, `BLOCKS`, `AFFECTS`, `NEEDS`, labelled by department	Filter on every edge's evidence
`dept_rollup`	Optional from and to department	Counts of open cross-team requests, blocks and at-risk items	Counts include visible edges only, so aggregates cannot leak
`impact_of`	Decision or change ID	Affected commitments, deals, projects, with owners	Per-recipient filter before anything is sent
`recall_memory` / `remember`	Kind, query or content	Agent memory entries	Namespace `(tenant, user, agent)`; item IDs re-checked on recall
`draft_email`	Intent, recipient, facts to use	Draft with a footnoted evidence list	Uses only facts the caller can see or that were cascaded as projections
`notify_owner`	Person, message, evidence IDs	Proposed Slack message	Propose, then confirm; internal notices may auto-send
`comment_github`	Issue or PR, text	Proposed comment	Propose, then confirm

Multi-hop reasoning. The answer agent runs a bounded plan-and-act loop, at most six tool calls per question.

1. Classify the intent (lookup, provenance, ownership, impact, draft) and expose only the tools that intent needs.
2. Each tool returns node IDs, so the next call starts from IDs, not from text.
3. The final answer includes the traversed path as a list of edges, shown in the UI as a mini graph.

4. A citation contract applies: the composer must attach evidence IDs to every claim, and a validator drops or flags any claim without one.

Example question: "Why is the Acme renewal at risk?" The loop walks the deal to the project it needs, to the decision blocking that project, to the Slack thread behind the decision, and cites that thread.

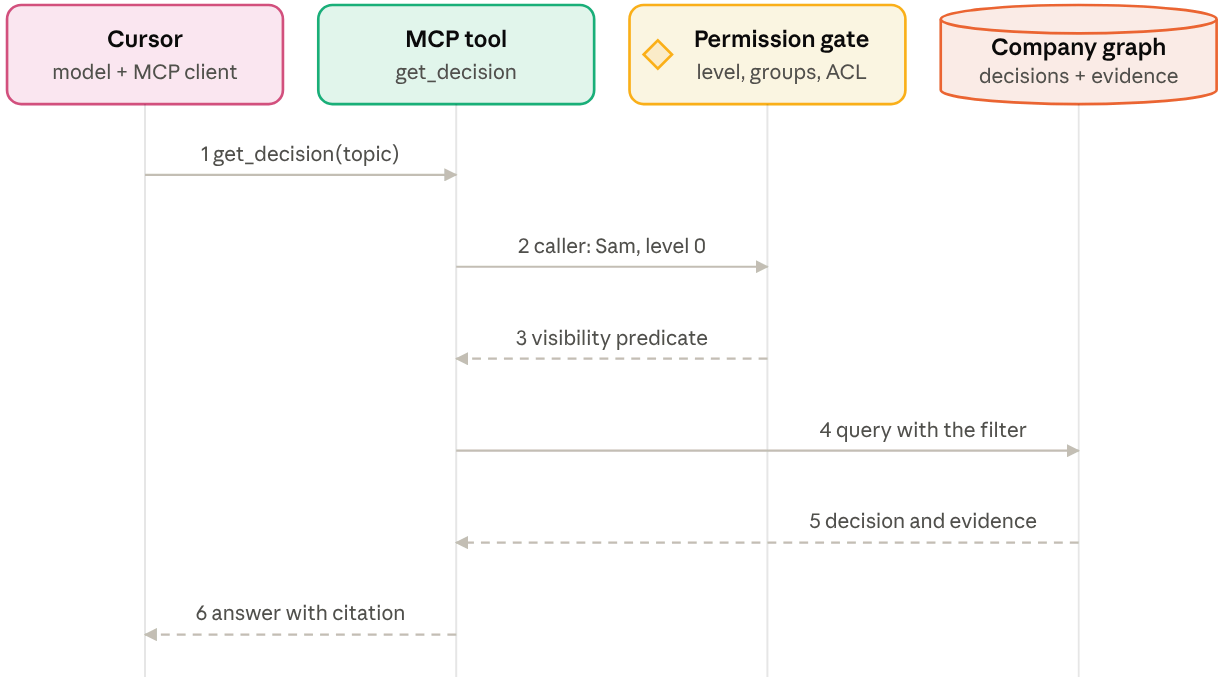
Tool selection. External clients such as Cursor see all tools through MCP `tools/list`. Descriptions are written as "use when" and "do not use when" so a client picks correctly. Our own agent narrows by intent first. We score selection with a 15-question set: right tool called, right citation returned.

Feature workflows

Three flows carry the demo, and each is a fixed sequence of tool calls, so every step can be tested on its own.

Flow 1. Cursor recommends the API the team chose (demo beat 3). Sam, an intern at level 0, starts the SSO endpoint. Cursor's model sees the `get_decision` description, calls it, and gets the current decision with only the evidence Sam may see.

Cursor gets the current decision and only the evidence Sam may see



Two outcomes change the answer
Superseded: the answer names the newer decision and says what it replaced.
No visible evidence: nothing is returned, and nothing hints that a source exists.

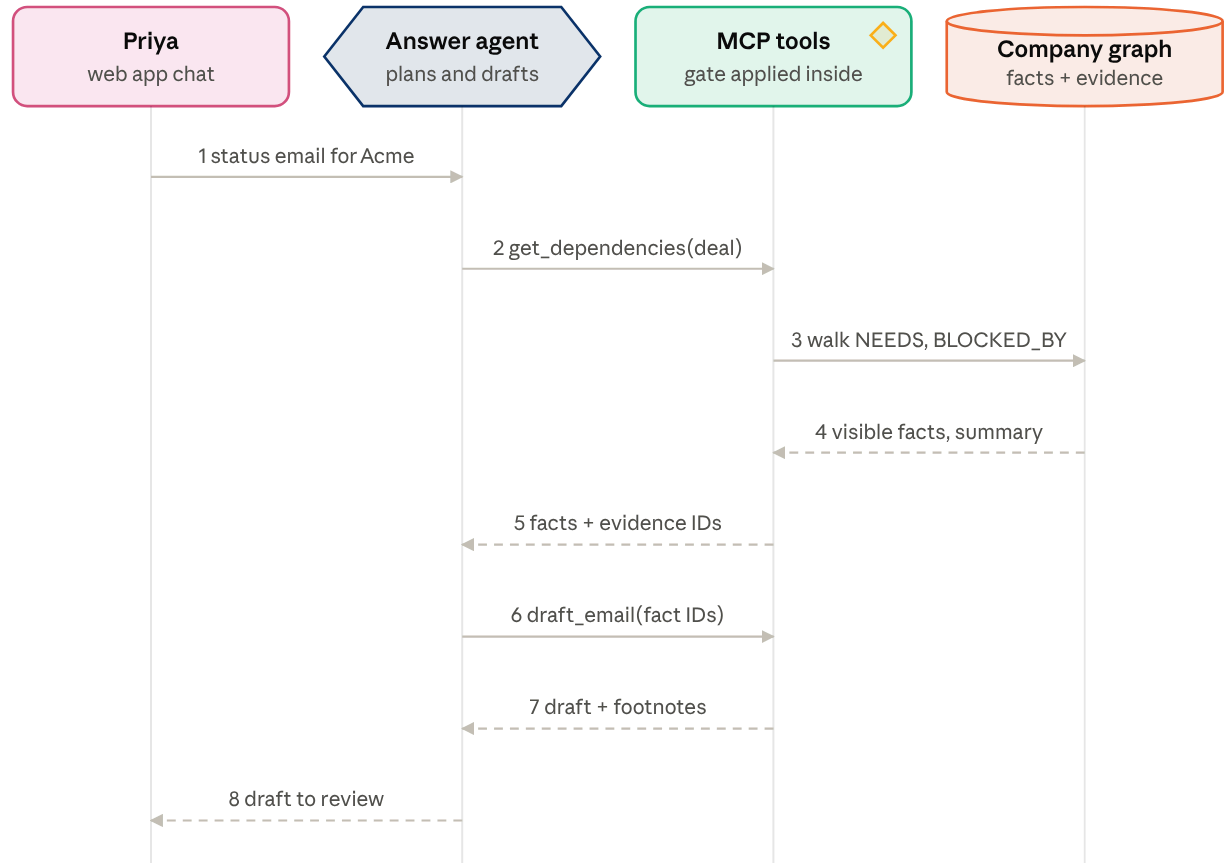
Solid: call. Dashed: result. Shapes follow the key in the system overview.

flow 1 · Cursor to graph, 6 calls, 1 permission check

Cursor points at our public MCP URL with Sam's token, and the tool never takes a user ID. The answer carries the quote, author, date and link to the Slack thread, so Sam can click through to the original. When two active decisions conflict, both are shown side by side. When none exists, the tool says so and suggests `search_context`.

Flow 2. A Sales email reflects the real engineering status (demo beat 6). Priya asks for a status update to Acme. The answer agent drafts from facts, not from memory: every sentence in the draft maps to an evidence ID Priya is allowed to see.

Every sentence of the status email maps to a fact Priya may see



What keeps the draft safe
The private engineering thread is invisible to Priya, so the draft cannot quote it.
Facts come from the GitHub status and the cascaded summary, each footnoted to its source.
The email is a draft: Priya reviews, edits and sends it herself.

Solid: call. Dashed: result. Shapes follow the key in the system overview.

flow 2 · status email, 8 calls, every fact footnoted

Priya cannot see the private engineering thread, so the draft uses the GitHub issue status and the summary that Engineering's channel cascaded. The footnote list names both sources. The draft is never sent: customer email stays draft-only (open decision 6), and Priya reviews and sends it herself.

Flow 3. A reversed decision alerts the people it hurts (demo beats 4 and 5). One Slack message from Dev starts the chain with no one asking a question: supersession, an impact walk, an owner notice and a drafted customer update.

flow 3 · reversal to alert, 10 calls, 1 handoff task

The impact walk follows `NEEDS` and `DEPENDS_ON` from the old decision to the commitments and deals that rest on it, and finds their owners. Priya owns the Acme deal, so she gets the DM. The notice is checked against the gate first, so she hears that her date is at risk without seeing the engineering thread. The handoff task carries IDs, and the answer agent builds the customer update from current facts, not from the alert text.

Freshness, explainability and citations

Knowledge stays current through incremental sync, supersession and re-verification, and every answer carries a citation block that a reviewer can follow back to the raw message.

What happens when things change. Sync uses a webhook where the tool offers one, and a polling cursor otherwise.

Event	How we detect it	Effect on the graph
Item created or edited	Webhook or cursor; `content_hash` differs	New item version, re-extraction, supersession check
Decision reversed	Extractor finds a new decision on the same subject and adjudication returns "supersedes"	Old decision gets `status = superseded` and `valid_to`; `SUPERSEDES` edge added; `ChangeEvent` emitted
Item deleted at source	Delete event, or missing on the reconcile pass	Evidence edges removed; a fact left with no evidence becomes `retracted`
Permission changed	ACL re-sync every 5 minutes, plus Slack membership and privacy events	Item `acl_groups` updated and the ACL epoch rises; applies from the next query, and no cached answer survives
Channel flips public or private	Slack privacy-change event, or the next 5-minute sync	Item and source ACLs change; no data moves, because visibility is checked at read time

Event	How we detect it	Effect on the graph
Sync stalls	Last good ACL sync older than 15 minutes	That source is excluded from every answer until the sync recovers; the observability panel shows ACL age
Figure grows old	`as_of` older than the staleness threshold	Answers show a stale badge and ask for a newer source
Fact repeated later	A newer item restates it	`last_verified_at` moves forward and confidence rises

Anatomy of an answer. The UI renders every answer as one card:

1. The answer, in one or two sentences.
2. Evidence per claim: a short quote, the author, the date, the source label and a link to the original.
3. The path the agent walked, drawn as a small graph of the edges it followed.
4. A freshness line, for example "Active since Oct 12, replaces the earlier choice of Provider A (Oct 3)".
5. Confidence: facts under the threshold are labelled "likely", and conflicting facts are shown side by side, never silently resolved.
6. A hidden-sources notice, off by default because a count of restricted sources reveals that they exist.

Every answer also writes one audit row: the caller, the surface, the tools called, the node IDs touched and the number of items filtered out.

Scale, tenancy and deployment

Isolation comes from one FalkorDB graph per tenant plus Postgres row-level security; throughput comes from batched writes, read-only queries and a small hot-read cache.

Concern	Design
Tenant isolation	Company graph `co:{tenant}`, private graphs `u:{tenant}:{user}`. Every Postgres row carries `tenant_id` under row-level security. The tenant comes from the token, never from a parameter. A second, tiny seeded tenant proves isolation in the demo.
Concurrent writes	Workers batch writes with `UNWIND`, 100 to 500 rows at a time. As far as I know FalkorDB runs one write at a time per graph, so each graph gets a single writer queue and different tenants write in parallel. Check this in the FalkorDB docs before relying on it.
Concurrent reads	Read-only queries over a connection pool. A short-lived cache holds hot lookups, keyed by the caller's visibility hash, the tool and its arguments.
Safe retries	Node IDs are deterministic hashes of type plus normalised key, and every write is a `MERGE`, so a retry changes nothing.
Slack events	The endpoint verifies Slack's signature, inserts the event ID into a table with a unique constraint, enqueues a job and returns 200 inside Slack's 3-second limit. A redelivered event hits the constraint and is dropped, and a per-thread lock keeps one reply per mention.
Backpressure	A Postgres `jobs` table with `SKIP LOCKED` is the queue. LLM calls pass a token bucket. A cheap classifier skips chunks that contain no decision, commitment or figure.
Auth	Per-user API tokens for MCP and the UI. A token and the surface resolve to a principal: tenant, user, level, groups, or the channel principal on a shared surface. The demo UI has a "view as" switcher.

Concern	Design
Observability	One request ID follows a question through every tool call. A small panel shows ingestion lag, queue depth, p95 query latency and ACL age per source.

Deployment (assumed). Docker Compose on one VM or a PaaS with five services: `api` (FastAPI and the MCP server), `worker` (ingestion and impact watcher), `web` (chat, graph explorer, view-as switcher), FalkorDB, and Postgres. The public URL serves `web` and the MCP endpoint, which Cursor points at.

Concurrency evidence. On day 5 we run a load script: about 50 simulated users asking questions while ingestion runs in the background. We record p95 latency and keep the numbers for the video.

Demo storyline and 5-day build plan

The demo seeds one invented company into the real Slack, Notion, Drive and GitHub, then tells a single story that touches every requirement: a promise made by Sales, a decision made by Engineering, a reversal, and a leadership priority that cascades down.

Cast (invented). Priya, Sales account executive, level 1. Rohan, Sales director, level 3. Dev, backend engineer, level 1. Maya, Engineering manager, level 2. Sam, Engineering intern, level 0. Customer: Acme Corp, renewing.

Beat	Who	What happens	What it proves
1. Promise	Priya	In a Slack deal channel she tells Acme that SSO ships by a given date; the Notion deal page and the Drive pricing sheet carry the renewal figures; a GitHub issue tracks SSO	Cross-source facts, Commitment and Figure extraction, `NEEDS` link from deal to project
2. Decision	Dev, Maya	In an engineering channel they choose Provider A for SSO	Decision node with evidence and owner

Beat	Who	What happens	What it proves
3. Recommendation	Sam	In Cursor, Sam starts the SSO endpoint and the agent recommends Provider A, citing the Slack thread	Flow 1, MCP, citation
4. Reversal	Dev	Dev posts that Provider A is out and Provider B is in	Supersession, `ChangeEvent`, freshness
5. Impact	Priya, Rohan	The impact watcher DMs Priya that the Acme date is at risk, then hands off to the answer agent, which drafts a customer update	Flow 3, cross-department edges, handoff, action
6. Status email	Priya	She asks for a status update to Acme; the draft uses the GitHub status and the cascaded summary, not the private engineering thread	Flow 2, projections, permission-aware drafting
7. Cascade	Maya, Sam	A private leadership channel sets a Q4 priority; Sam asks what to focus on and the agent cites the summary	Hierarchical visibility with summary-only cascade
8. Boundary	Sam, Maya	Sam asks for a leadership budget figure and gets nothing; Maya gets it with citations, but in a channel Sam belongs to she gets only what Sam may see	Source permissions, view-as switcher, audience-aware replies

Cut order if we run late. Drop these first, in this order: the second tenant, the hidden-sources notice, the `comment_github` tool, automatic playbook suggestions, and the `KNOWS_ABOUT` recompute job (replace it with plain authorship counts).

Five-day build plan. Two builders cover the system in five days, and a feature freeze at the end of day 4 leaves day 5 for deploy, the load test and the video.

build plan · 2 builders, 5 days, 13 bars, 1 feature freeze

Builder A and Builder B are placeholders for the two of you. Each bar is a thin first version. A bar in the Both lane overlaps each builder's own bar: you stop and do that work together. Day 2 (extraction, resolution and the permission test cases) and day 4 (agents and the web app) carry the most risk, so if either slips, apply the cut order above before touching the freeze.

Day	Done when
1	Raw items from all four sources sit in Postgres, and the graph schema is loaded with the seeded org chart, and the Slack endpoint acks inside Slack's 3-second limit and drops duplicate events
2	A Slack thread produces a Decision with evidence, and the visibility tests pass for all five people in the matrix, plus the channel-audience, privacy-flip, revocation, stale-snapshot and authority cases
3	Cursor gets a cited decision through MCP, and a reversed decision supersedes the old one and emits a ChangeEvent, and the test that no agent tool can write the graph passes
4	All eight demo beats run end to end on the seeded company; feature freeze
5	The deployed link is live, the load-test numbers are recorded, and the README and the video are done

Open decisions

Eleven decisions remain; each has a default in this document, so if we say nothing we build the default.

#	Decision	Default in this document	Alternative
1	What management-level visibility means for an intern's agent	Summaries of decisions that affect them cascade down; raw management sources stay restricted	Interns see raw management sources (lower `min_level`)
2	Who creates the summaries	Automatic for channels flagged `broadcast`; a manager approves everything else	Always manager-approved
3	Stack	Python and FastAPI, Postgres with pgvector, FalkorDB, MCP over HTTP	Your own stack
4	Org chart source	A seeded file with reporting line, level and department	Read it from a Notion page
5	Hidden-sources notice	Off, because a count reveals that restricted sources exist	On, with a request-access flow
6	Real outbound actions in the demo	Internal Slack DM sends for real; GitHub comment and customer email stay draft-only	Everything draft-only
7	Second tenant	Tiny seeded tenant, first thing cut if late	Skip it
8	Email and Jira	Out of v1	Add one if time allows
9	Replies in shared Slack channels	A channel principal (lowest level among members), with the full answer offered by DM	The bot answers only in DMs
10	Source authority ranks	Formal documents 3, trackers 2, chat 1; a lower rank cannot supersede a higher one unless its author made the old claim	The newest claim always wins

#	Decision	Default in this document	Alternative
11	ACL sync interval and stale cutoff	Every 5 minutes; a source is excluded after 15 minutes without a good sync	A shorter interval, or fail open with a warning